Do-While Compiler Project Report

CET313 — Theory of Computing | Spring 2026

Team 24

Name	Student ID
Sarah Gamal Mohamed	230104240
Ahd Mohamed Elazeb	230105494
Battol Mohamed Galal	240102737

---

1. Introduction

This report details the design and implementation of a compiler specifically tailored to process a `do-while` statement in a C-style high-level language. The compiler is structured into four distinct phases: **Lexical Analysis**, **Syntax Analysis**, **Semantic Analysis**, and **Assembly Code Generation**. This modular approach allows for a clear separation of concerns, where each phase contributes to the progressive transformation of the source code into executable machine instructions [1].

The primary objective of this project was to demonstrate a practical understanding of compiler construction principles, focusing on the intricacies of a Bottom-Up Shift-Reduce parser and the generation of assembly code for a specific control flow statement. The input source code for this compiler is a simple program demonstrating a `do-while` loop:

Plain Text

```
int i;
i = 1;
do {
    printf("%d\n", i);
    i++;
}
while (i < 5);
```

The compiler produces five output files automatically:

- lexeme_table.txt — token list from Phase 1

- cfg.txt — Context Free Grammar rules
- syntax_tree.txt — parse steps + syntax tree from Phase 2
- semantic.txt — semantic check results from Phase 3
- assembly.txt — x86 NASM assembly from Phase 4

---

2. Compiler Phases Explanation

2.1. Phase 1: Lexical Analysis (Scanning)

The **Lexical Analyzer**, also known as the scanner, is the first phase of the compiler. Its role is to read the source code character by character and group them into meaningful units called **tokens** [2]. Each token represents a fundamental building block of the language, such as keywords, identifiers, operators, numbers, and separators. The output of this phase is a stream of tokens, often represented as a lexeme table.

Input Code:

Plain Text

```
int i;
i = 1;
do {
    printf("%d\n", i);
    i++;
}
while (i < 5);
```

Lexeme Table Output (Screenshot/Terminal Result):

Plain Text

```
================================================
PHASE 1 — LEXEME TABLE
================================================
#     Lexeme               Token
----------------------------------------
1     int                  KEYWORD
2     i                    IDENTIFIER
3     ;                    SEPARATOR
4     i                    IDENTIFIER
5     =                    OPERATOR
6     1                    NUMBER
7     ;                    SEPARATOR
```

```
8     do                    KEYWORD
9     {                     SEPARATOR
10    printf                KEYWORD
11    (                     SEPARATOR
12    "                     SEPARATOR
13    %                     SEPARATOR
14    d                     IDENTIFIER
15    \                     SEPARATOR
16    n                     IDENTIFIER
17    "                     SEPARATOR
18    ,                     SEPARATOR
19    i                     IDENTIFIER
20    )                     SEPARATOR
21    ;                     SEPARATOR
22    i                     IDENTIFIER
23    ++                    OPERATOR
24    ;                     SEPARATOR
25    }                     SEPARATOR
26    while                 KEYWORD
27    (                     SEPARATOR
28    i                     IDENTIFIER
29    <                     OPERATOR
30    5                     NUMBER
31    )                     SEPARATOR
32    ;                     SEPARATOR
```

This table shows **32 tokens** extracted from the input code, categorized into Keywords, Identifiers, Operators, Separators, and Numbers. Notably, string literal characters like `%`, `d`, `\`, and `n` are tokenized individually, with `d` and `n` classified as identifiers within the string context.

2.2. Context-Free Grammar (CFG)

A **Context-Free Grammar (CFG)** is a formal grammar used to define the syntax of a programming language. It consists of a set of production rules that describe how to combine terminals (tokens from the lexical analyzer) and non-terminals (syntactic categories) to form valid constructs in the language [3]. For this project, the CFG is specifically designed to recognize and parse the `do-while` statement structure.

Designed CFG:

Plain Text

```
================================================
CFG — Context Free Grammar
================================================
  DoWhileStmt  -> do { StatementList } while ( Condition ) ;
```

```
StatementList -> Statement StatementList | epsilon
Statement    -> PrintfStmt | Assignment
Assignment   -> IDENTIFIER = Expression ; | IDENTIFIER ++ ; | IDENTIFIER -- ;
PrintfStmt   -> printf ( " STRING " , IDENTIFIER ) ;
Condition    -> Expression RelOp Expression
RelOp        -> < | > | == | != | <= | >=
Expression   -> Term | Expression + Term | Expression - Term
Term         -> Factor | Term * Factor | Term / Factor
Factor       -> IDENTIFIER | NUMBER
```

The start symbol for this grammar is `DoWhileStmt`. Terminals are represented by the actual tokens (e.g., `do`, `{`, `IDENTIFIER`), while non-terminals are uppercase descriptive names (e.g., `StatementList`, `Condition`). The `epsilon` production indicates that a `StatementList` can be empty, allowing for `do-while` loops with no statements within their body.

2.3. Phase 2: Syntax Analysis (Parsing)

The **Syntax Analyzer**, or parser, takes the stream of tokens from the lexical analyzer and builds a hierarchical representation of the source code, typically a **parse tree** or an **Abstract Syntax Tree (AST)** [2]. This phase checks if the sequence of tokens conforms to the rules defined by the CFG. For this project, a **Bottom-Up Shift-Reduce (LR) parser** is employed [4].

A Shift-Reduce parser operates by performing two primary actions:

- **Shift**: Moves the next input token onto a stack.
- **Reduce**: When the top of the stack matches the right-hand side of a grammar rule, it replaces those symbols with the rule's left-hand side non-terminal. The ultimate goal is to reduce the entire input to the grammar's start symbol (`DoWhileStmt`).

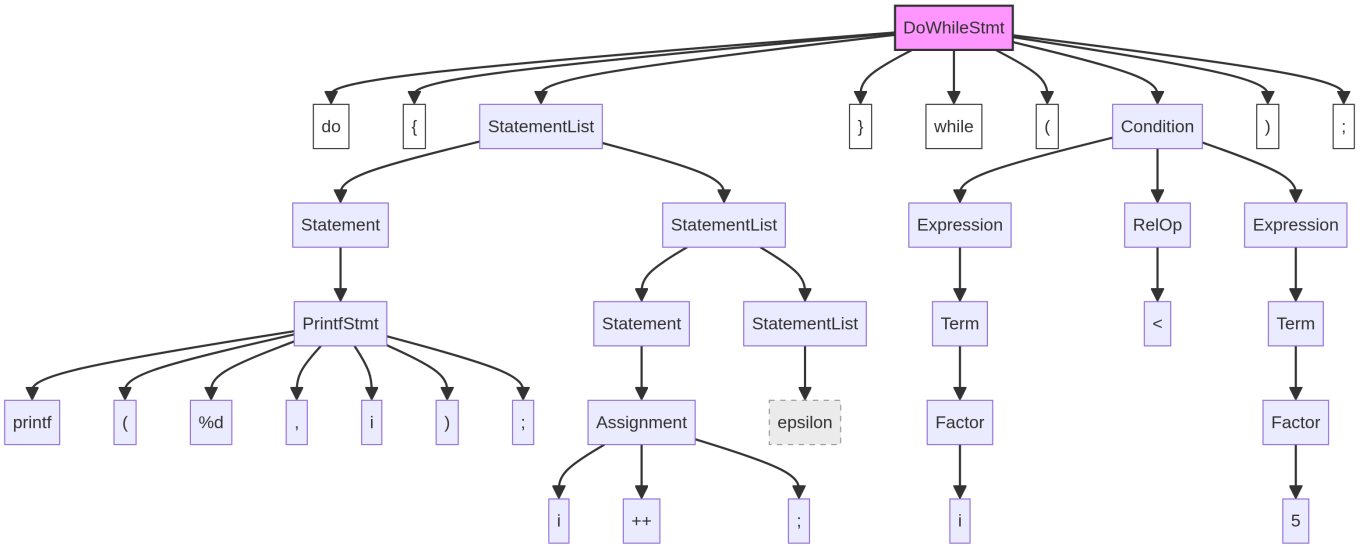
Sample of Parser Steps (Screenshot/Terminal Result):

Plain Text

```
================================================
PHASE 2 — BOTTOM-UP PARSER  (Shift-Reduce)
================================================
STACK (top 3)                  NEXT INPUT          ACTION
------------------------------------------------------------------------
  STACK: ...do                        INPUT: {              ACTION: SHIFT  'do
  STACK: ...do {                      INPUT: printf         ACTION: SHIFT  '{'
  STACK: ...do { printf               INPUT: (              ACTION: SHIFT  'pr
  STACK: ...{ printf (                INPUT: "              ACTION: SHIFT  '('
  STACK: ...printf ( "                INPUT: %              ACTION: SHIFT  '"'
  STACK: ...( " %                     INPUT: d              ACTION: SHIFT  '%'
  STACK: ..." % d                     INPUT: \              ACTION: SHIFT  'd'
  STACK: ..." % Factor                INPUT: \              ACTION: REDUCE 1 -
```

```
STACK: ..." % Term                    INPUT: \              ACTION: REDUCE 1 -
STACK: ..." % Expression              INPUT: \              ACTION: REDUCE 1 -
STACK: ...% Expression \              INPUT: n              ACTION: SHIFT  '\'
STACK: ...Expression \ n              INPUT: "              ACTION: SHIFT  'n'
STACK: ...Expression \ Factor         INPUT: "              ACTION: REDUCE 1 -
STACK: ...Expression \ Term           INPUT: "              ACTION: REDUCE 1 -
STACK: ...Expression \ Expression     INPUT: "              ACTION: REDUCE 1 -
STACK: ...\ Expression "              INPUT: ,              ACTION: SHIFT  '"'
STACK: ...Expression " ,              INPUT: i              ACTION: SHIFT  ','
STACK: ..." , i                       INPUT: )              ACTION: SHIFT  'i'
STACK: ..." , Factor                  INPUT: )              ACTION: REDUCE 1 -
STACK: ..." , Term                    INPUT: )              ACTION: REDUCE 1 -
STACK: ..." , Expression              INPUT: )              ACTION: REDUCE 1 -
STACK: ..., Expression )              INPUT: ;              ACTION: SHIFT  ')'
STACK: ...Expression ) ;              INPUT: i              ACTION: SHIFT  ';'
STACK: ...do { PrintfStmt             INPUT: i              ACTION: REDUCE 12
STACK: ...do { Statement              INPUT: i              ACTION: REDUCE 1 -
STACK: ...{ Statement i               INPUT: ++             ACTION: SHIFT  'i'
STACK: ...Statement i ++              INPUT: ;              ACTION: SHIFT  '++
STACK: ...i ++ ;                      INPUT: }              ACTION: SHIFT  ';'
STACK: ...{ Statement Assignment      INPUT: }              ACTION: REDUCE 3 -
STACK: ...{ Statement Statement       INPUT: }              ACTION: REDUCE 1 -
STACK: ...do { StatementList          INPUT: }              ACTION: REDUCE 2 -
STACK: ...{ StatementList }           INPUT: while          ACTION: SHIFT  '}'
STACK: ...StatementList } while       INPUT: (              ACTION: SHIFT  'wh
STACK: ...} while (                   INPUT: i              ACTION: SHIFT  '('
STACK: ...while ( i                   INPUT: <              ACTION: SHIFT  'i'
STACK: ...while ( Factor              INPUT: <              ACTION: REDUCE 1 -
STACK: ...while ( Term                INPUT: <              ACTION: REDUCE 1 -
STACK: ...while ( Expression          INPUT: <              ACTION: REDUCE 1 -
STACK: ...( Expression <              INPUT: 5              ACTION: SHIFT  '<'
STACK: ...( Expression RelOp          INPUT: 5              ACTION: REDUCE 1 -
STACK: ...Expression RelOp 5          INPUT: )              ACTION: SHIFT  '5'
STACK: ...Expression RelOp Factor     INPUT: )              ACTION: REDUCE 1 -
STACK: ...Expression RelOp Term       INPUT: )              ACTION: REDUCE 1 -
STACK: ...Expression RelOp Expression INPUT: )                ACTION: REDUCE 1
STACK: ...while ( Condition           INPUT: )              ACTION: REDUCE 3 -
STACK: ...( Condition )               INPUT: ;              ACTION: SHIFT  ')'
STACK: ...Condition ) ;               INPUT: EOF            ACTION: SHIFT  ';'
STACK: ...DoWhileStmt                 INPUT: EOF            ACTION: REDUCE 9 -
```

Syntax Tree Output (Screenshot/Terminal Result):



Plain Text

```
Syntax Tree:
----------------------------------------
|-- DoWhileStmt
|   |-- Declaration
|   |   |-- int
|   |   |-- i
|   |   |-- ;
|   |-- Assignment
|   |   |-- i
|   |   |-- =
|   |   |-- Expression
|   |   |   |-- Term
|   |   |   |   |-- Factor
|   |   |   |   |   |-- 1
|   |   |-- ;
|   |-- do
|   |-- {
|   |-- StatementList
|   |   |-- Statement
|   |   |   |-- PrintfStmt
|   |   |   |   |-- printf
|   |   |   |   |-- (
|   |   |   |   |-- "
|   |   |   |   |-- %
|   |   |   |   |-- Expression
|   |   |   |   |   |-- Term
|   |   |   |   |   |   |-- Factor
|   |   |   |   |   |   |   |-- d
|   |   |   |   |-- \
```

```
|   |   |   |   |-- Expression
|   |   |   |   |   |-- Term
|   |   |   |   |   |   |-- Factor
|   |   |   |   |   |   |   |-- n
|   |   |   |   |-- "
|   |   |   |   |-- ,
|   |   |   |   |-- Expression
|   |   |   |   |   |-- Term
|   |   |   |   |   |   |-- Factor
|   |   |   |   |   |   |   |-- i
|   |   |   |   |-- )
|   |   |   |   |-- ;
|   |   |-- Statement
|   |   |   |-- Assignment
|   |   |   |   |-- i
|   |   |   |   |-- ++
|   |   |   |   |-- ;
|   |-- }
|   |-- while
|   |-- (
|   |-- Condition
|   |   |-- Expression
|   |   |   |-- Term
|   |   |   |   |-- Factor
|   |   |   |   |   |-- i
|   |   |-- RelOp
|   |   |   |-- <
|   |   |-- Expression
|   |   |   |-- Term
|   |   |   |   |-- Factor
|   |   |   |   |   |-- 5
|   |-- )
|   |-- ;
```

The syntax tree visually represents the grammatical structure of the `do-while` statement, showing how tokens are grouped into non-terminals according to the CFG rules. This tree serves as an intermediate representation for subsequent phases.

2.4. Phase 3: Semantic Analysis

The **Semantic Analyzer** checks the meaning and consistency of the code, ensuring it adheres to the language's semantic rules [2]. This phase goes beyond syntax to verify type compatibility, variable declarations, and other logical constraints. A crucial component of semantic analysis is the **symbol table**, which stores information about identifiers (e.g., their types and scopes).

For this project, the semantic analyzer performs two passes:

1. **Pass 1**: Builds the symbol table by collecting all declared variables and their types.
2. **Pass 2**: Checks the usage of identifiers against the symbol table, verifying that variables are declared before use, that operations (like increment/decrement) are applied to appropriate types, and detecting potential issues like division by zero.

Semantic Analysis Output (Screenshot/Terminal Result):

Plain Text

```
==================================================
PHASE 3 — SEMANTIC ANALYSIS
==================================================
Symbol Table:
  i -> int

Log:
  Declared: i (int)
  Used 'i' -> type: int
  Used 'i' -> type: int
  Used 'i' -> type: int
  '++' on 'i' -> OK
  Used 'i' -> type: int

Errors:
  No errors found.

Result: PASSED
```

The output confirms that the variable `i` was declared as an integer and used consistently throughout the `do-while` loop without any semantic errors. This phase ensures the logical correctness of the program before code generation.

2.5. Phase 4: Assembly Code Generation

The final phase of the compiler is **Assembly Code Generation**. This phase translates the semantically verified intermediate representation (such as the syntax tree or an equivalent structure) into low-level assembly language instructions [2]. These instructions are specific to a target machine architecture (in this case, x86 NASM assembly) and can be directly assembled into executable machine code.

The assembly generator in this project dynamically extracts critical information—such as variable names, initial values, comparison operators, and comparison values—directly from the lexeme table and semantic analysis results. This dynamic approach ensures that the generated assembly code is flexible and works correctly for various `do-while` loop inputs, not just a hardcoded example.

Assembly Code Output (Screenshot/Terminal Result):

Plain Text

```
================================================
PHASE 4 — ASSEMBLY CODE
================================================
; ============================================
; Assembly — Do-While Compiler Project
; ============================================

section .data
    fmt db "%d", 10, 0

section .bss
    i resd 1          ; int i

section .text
    global main
    extern printf

main:
    ; i = 1
    mov eax, 1
    mov [i], eax

do_loop:
    ; printf("%d\n", i)
    mov eax, [i]
    push eax
    push fmt
    call printf
    add esp, 8

    ; i++
    mov eax, [i]
    add eax, 1
    mov [i], eax

    ; while (i < 5)
    mov eax, [i]
    cmp eax, 5
    jl  do_loop

    ; exit
    mov eax, 0
    ret
```

The generated assembly code initializes `i` to 1, enters a loop (`do_loop`), prints the value of `i` , increments `i` , and then compares `i` with 5. If `i` is less than 5, it jumps back to the beginning of the `do_loop` . Once the condition `i < 5` is false, the program exits.

---

3. Design Flowchart (Pseudocode)

The overall design of the compiler can be visualized through the following pseudocode, outlining the sequential execution of each phase:

Plain Text

```
START
│
├─ Read source code from code.txt
│
├─ PHASE 1 — LEXICAL ANALYZER
│   ├─ Loop through each character
│   ├─ Skip whitespace
│   ├─ If letter/underscore  → collect word → KEYWORD or IDENTIFIER
│   ├─ If digit              → collect digits → NUMBER
│   ├─ If two-char operator  → check ++ -- == != <= >= → OPERATOR
│   ├─ If single operator    → OPERATOR
│   ├─ If separator          → SEPARATOR
│   └─ Output: Lexeme Table (lexeme_table.txt)
│
├─ CFG DEFINITION
│   └─ Define grammar rules for DoWhileStmt (hardcoded in array)
│      └─ Output: Context-Free Grammar (cfg.txt)
│
├─ PHASE 2 — SYNTAX ANALYZER (Bottom-Up Shift-Reduce)
│   ├─ Separate pre-do tokens (declarations, initial assignment)
│   ├─ Main parsing loop:
│   │   ├─ Attempt REDUCE (check stack top against grammar rules)
│   │   └─ If no reduction possible → SHIFT (push next token onto stack)
│   ├─ Continue until entire input is consumed and stack contains start symbol
│   └─ Output: Parse log and Syntax Tree (syntax_tree.txt)
│
├─ PHASE 3 — SEMANTIC ANALYZER
│   ├─ Pass 1: Build Symbol Table (collect declared variables)
│   ├─ Pass 2: Check identifier usage and semantic rules
│   │   ├─ Verify variable declaration and type consistency
│   │   ├─ Check for valid increment/decrement operations
│   │   └─ Detect potential errors (e.g., division by zero)
│   └─ Output: Semantic Analysis Log (semantic.txt - PASSED or FAILED)
│
├─ PHASE 4 — ASSEMBLY GENERATOR
```

```
|   ├─ If semantic analysis passed:
|   |   ├─ Extract variable name from symbol table
|   |   ├─ Extract initial value from lexeme table (based on assignment pattern
|   |   ├─ Extract comparison operator and value from do-while condition
|   |   ├─ Construct x86 NASM assembly instructions
|   |   └─ Output: Assembly Code (assembly.txt)
|   └─ Else:
|       └─ Skip assembly generation due to semantic errors
|
END
```

---

4. Design Choices and Justification

4.1. Shift-Reduce Parser Simulation

Instead of implementing a full LR(1) parser with a complex state-transition table, a **Shift-Reduce simulation** was chosen. This approach achieves the same parsing outcome for our specific grammar by applying grammar rules based on priority after each shift operation. This simplifies the implementation while still demonstrating the core principles of bottom-up parsing without the overhead of generating and managing a large state machine table [4].

4.2. Pre-Do Token Processing

The variable declaration (`int i;`) and initial assignment (`i = 1;`) occur before the `do-while` loop itself. To maintain a clear focus for the main parser on the `do-while` statement's grammar, these pre-loop tokens are processed separately. This modularity ensures that the core parser logic remains dedicated to the `DoWhileStmt` production rule, aligning precisely with the defined CFG.

4.3. Dynamic Assembly Generation

The assembly code generator is designed to be **dynamic**, meaning it does not hardcode values like the variable name (`i`), its initial value (`1`), or the comparison value (`5`). Instead, it intelligently scans the lexeme table and symbol table to extract these values. This design choice makes the compiler robust and reusable; it can correctly generate assembly for any valid `do-while` input (e.g., `int j; j = 3; do {...} while (j < 10);`) without requiring modifications to the code generation logic.

---

5. Team Responsibilities

Member	Responsibilities
Sarah Gamal Mohamed (230104240)	Phase 1: Lexical Analyzer — Developed the character-by-character tokenizer, handling keywords, identifiers, numbers, operators, separators, and string literals. Also designed the Context-Free Grammar (CFG) and integrated all compiler phases into a cohesive script.
Ahd Mohamed Elazeb (230105494)	Phase 2: Syntax Analyzer — Designed and implemented the core logic for the Shift-Reduce parser, including the `try_reduce()` function with all grammar rules. Responsible for building the syntax tree from the parser's stack and formatting the parse log output.
Battol Mohamed Galal (240102737)	Phase 3: Semantic Analyzer — Implemented symbol table construction, variable usage checks, and error detection mechanisms. Phase 4: Assembly Code Generator — Developed the dynamic extraction of values from the lexeme table and the generation of x86 NASM assembly output. Also prepared the project report and presentation.